7xtgv
History Guide

Table of Contents

1. Reading this reconstruction
2. Where it began
3. Why this project
4. How it evolved
5. The native format: what changed, and what is still only designed
6. Review prompts for the final SPEC
7. Related technical history

Raw reconstruction for our own review and memory recovery, assembled 261013. This deliberately retains details, detours and unfinished ideas; it is material for a later shorter history, and a cross-check for the native specification.

The central thread is a small tiled display engine for the wider 7xsys world: readable text, syntax-coloured editors, windows and instruments, with modest memory and simple host access. The working RunCPM terminal, physical boards, desktop ports and browser are substantial development vehicles. They do not mean that the proposed four-byte native 7xtgv engine has been implemented.

1. Reading this reconstruction

- **Recorded idea** means that a source contains the proposal, not that we built it. **Reported test** means a dated discussion or checkpoint reports an observation; it is not a new test performed for this history.
- The private ChatGPT extracts include retrospective reconstructions, later annotations, mistaken assistant answers and corrections. We retain useful memory cues while explicitly separating these from established implementation.
- Dates in filenames, dates recalled inside a conversation and recovery dates can differ. An archive label alone cannot establish when a feature worked.
- Private discussions are cited by description only; their documents and ZIP archives are not linked from this guide. Public repository references remain linked. Source archives remain unchanged and are not included in Git.
- Our recollection supplied on 261013 is that **2506xx through 2609xx was mostly occupied by 7xsys**, interrupted by a few unusual events. `251101` remains a private personal marker, with no event details inferred here. `250515` is the supplied date for **nukedc**: accidentally nuking our own Windows C drive. Both precise markers are retained as given; 250515 precedes that approximate interval. The recovered loss narrative also involves January 2026 work and a 2026 backup gap, so its date relationship remains unresolved.
- Earlier work involved considerable context switching. September through this reconstruction was, by our present recollection, sustained focus on this development. That helps explain why the recent daily record is much denser.

2. Where it began

250529
the original TILED-GUI-VIDEO-256 idea

Sources: original concept preserved in the September archive and private copy with surrounding context.

- The starting point was a **fast, simple tiled GUI**, especially an editor with useful syntax colours, rather than a sprite-heavy games machine. A small host should update character and attribute memory instead of repeatedly painting a full bitmap. Window frames and controls should use the same cheap drawing vocabulary as text.
- The surrounding working world included Sinclair QL, QPC2/Qemulator, SMSQ/E and S*Basic, the QLC commander idea, and **LSB, line-number stripped BASIC**. The editor was part of a larger software environment; VMEX/ASLIX appears in that context, not as a completed subsystem underneath the demo.
- The intended hosts were broader than Z80: the notes explicitly span small 8-, 16- and 32-bit systems and the **7xsys / 7xmod / 7xmem** hardware family. A useful display component should remain separable from the chosen CPU.
- The original **7xbus MIPI16** wording belongs to that hardware discussion, alongside RGB565 transfers to LCDs. It should not silently be translated into a claim that we had specified a modern MIPI DSI link or finished its pinout.
- This direction explains the repeated attraction to character RAM, compact attributes, writable glyphs, palettes and simple bus transactions. It also explains why a powerful ready-made graphics solution could still miss the desired programming model.

The first cell, font and palette budget

- The early draft used one character/tile byte and one attribute byte per cell. The character byte carried a seven-bit character number plus a bank-selector bit; two upper attribute bits supplied the remaining bank selection. Three PAPER and three INK bits selected colours.

- Eight banks of 128 glyphs, with eight stored row bytes each, gave an 8192-byte font budget. This arithmetic belongs to the **6x8-era draft**. With ten row bytes the same organisation would require 10240 bytes; the raster change cannot be made without revisiting the old memory budget.
- A rough 16-KiB allocation was discussed as approximately 4 KiB of tiles, 4 KiB of attributes and 8 KiB of fonts. It was a design sketch, not a settled register or address map. Eight sets of sixteen palette indices would add a small 128-byte selection table.
- A programmable **256-colour LUT** was an explicit ambition. HSV-like schemes of sixteen hues by sixteen levels, or thirty-two by eight, were considered. Fixed RGB332 was not the intended final colour vocabulary.
- The two unused bits beside six active glyph pixels were considered for per-row ink/paper brightness information. This was an early packing idea; the later native design reserves those low row bits and moves brightness into EXTA. These are different stages, not simultaneous requirements.
- Raster and grid calculations were still exploratory. References to 6x8, 6x10, 96x60 and 80x48 belong to that exploration. One comparison says 512x348 for a 64x48 array of 8-pixel tiles; the arithmetic would be 512x384. That typo is evidence of working notes, not a display timing we adopted.

What the screen was meant to feel like

- Positive paper and readable small text mattered. A night palette could transform the appearance without rewriting all the cells.
- Ordinary text could be dark grey and highlighted or “bold” text black. At this tiny raster, making a stroke physically thicker can destroy a glyph; colour/intensity was another way to convey emphasis.
- The motivating editor examples included green comments, red literals, blue operators and black keywords across languages. These were UI intentions, not a claim that a complete syntax-highlighting editor existed.
- Banks 0/1 were imagined for regular/bold text, 2/3 for window and control glyphs, including alternate LED/control states, and 4..7 for bitmap panes.

- Shared writable glyphs immediately raised a window problem: redefining a pattern for the foreground can damage an inactive window using that pattern. Foreground redraw and resource ownership were already concerns.
- Readability at **2x scale** on small displays was a practical objective. Fine/vertical scrolling, a cursor, and a pointer were also considered. Alternating glyphs or attributes could provide a simple blinking cursor; a hardware sprite cursor was not automatically worth the complexity.
- Transparency and game facilities were possible extensions. They did not replace the editor/windowing purpose that motivated the architecture.

Early hardware alternatives

- **Zeal 8-bit video** was found after the initial concept. Its simpler text mode was attractive as a possible route while retaining its games machinery separately. Using a Zeal emulator as a prototype environment was considered. This was exploration, not evidence of a completed port.
- Bill Shen/plasmo's **VGARC64x48** was a useful comparison for compact VGA tile output, with the raster arithmetic caveat above.
- A **CPLD plus dual-port SRAM**, with roughly 16 or 32 KiB, offered a deliberately small hardware implementation. An **SAA7105H** with indexed input, a 24-bit LUT and PAL/NTSC output entered the discussion.
- The original question about VGA pass-through and a 27.125-MHz clock remained a question. This history does not certify those electrical/timing assumptions. Later FPGA palette/RGB565 conversion or an R-2R VGA DAC were other directions.
- The SAA pointer facility, including a 32x32 pointer discussed in the note, was compared with simpler software/attribute cursor choices. No completed SAA-based 7xtgv board is established by these sources.

3. Why this project

The wider 7xsys hardware and software context

Sources: original system context, recovery discussion, host and bridge research, interpreter memory lessons.

- **7xsys** is the broader system-building context. Its long-running hardware work explains why a reusable video subsystem kept returning between more immediate tasks. Our 261013 recollection places most of 2506xx–2609xx in this wider work; the 7xtgv excerpts are samples of it, not a complete 7xsys diary.
- **7xmod** appears with the modular hardware ambitions in the origin proposal. This source set does not establish a complete module catalogue, connector revision history or finished bill of materials. Keeping the name visible is useful for finding the missing surrounding design records later.
- **7xmem** appears both in the original family and in the recovery search: **2x40-pin** and **80+GND SODIMM** arrangements are explicit memory cues. **MiSTer XSDS** also appears in that discussion. These are evidence that those designs were in our working context, not proof of a recovered final pinout. An assistant's example recovery table saying a pinout was confirmed does not itself establish that the corresponding artifact survived.
- **7xbus** was the host/display boundary under consideration. Repeated searches for a simple parallel interface, 3.3-V compatibility and small external RAM make sense within that boundary. The precise final bus transaction contract remains a separate specification task.
- **VMEX/ASLIX** connects the editor/language environment to a future portable execution environment. The January RP2350 conversation mentions an **RV32EM-like** direction amid frustration with build/upload cycles. The extracts do not provide a completed ISA or runtime specification; we must not manufacture one from the abbreviation.
- The recurring wish was to iterate programs and UI quickly across hardware, with a stable display model and useful native services. Long compiler cycles, changing graphics APIs and board-specific bring-up were practical reasons to want that separation.

- Another VMEX/ASLIX trace surfaced during the 261007 licensing review: the reserved-rights wording mistakenly copied into font/demo headers was intended for VMEX/ASLIX only. Removing it from these headers, followed by the terminal MIT decision, did not change those separate VMEX/ASLIX notices. Recorded correction
- CP/M, FUZIX, BASICs, monitors and the planned VM environment were possible clients. RunCPM eventually supplied real workloads and repeatable tests; it did not retroactively become the sole reason for inventing 7xtgv.

Memory and interpreter lessons carried forward in September

- The 260909 memory note explicitly treats RunCPM as a proxy for VMEX questions: interpreter/hot state, native services and objects, peripheral I/O, and larger external capacity such as PSRAM.
- Putting one function in IRAM does not move every called function there, and IRAM allocation consumes scarce internal memory. Moving the roughly 29-KiB Z80 execution body and small BIOS/BDOS dispatch pieces had to be judged by measured benefit; relocation alone was not a universal answer.
- Hot bytecode, stack and metadata were candidates for internal RAM; bulk objects, cold data and framebuffers were candidates for PSRAM. Paged or procedure-level caches remained future options justified only by measurement.
- Repeated open/seek/close work around 128-byte guest records was another warning: storage overhead can dominate a VM benchmark. Later Windows handle-reuse work gave a concrete instance of this lesson. It does not mean the proposed VMEX storage/cache architecture has been implemented.
- The DDT/history plan explicitly includes future VMEX/ASLIX CLIs, monitors and database REPLs as possible users of local composition and history. Full-screen or immediate-character programs need a different input boundary. DDT, history and input-origin plan
- Favourites, macros, keyword lists, paste pacing and scripts were distinct proposals. Their appearance in a history plan must not be read as evidence that today’s history engine implements all of them.

QL influence
more than copying a font

- The compact font, positive-paper editor, colour semantics, channels and commander-style navigation form a connected set of influences. Their recurrence helps explain why a generic 8x16 terminal was not a full answer.
- The 261007 research revisits QDOS channels as stateful interfaces carrying cursor, colour and font/window-related state. It separates that software interface from a hardware tile engine and from an automatic window manager. QL channels research and native sidenote
- The later **XC** commander idea belongs in the memory map too: a portable BDOS application with state saved before launching another COM, and a route back through the CCP. Because a COM overwrites the transient program area, a resident application cannot simply be assumed to survive.
- The 260926 proposal considers a CCP-specific `$$$.SUB` return sequence with reversed 128-byte records, and an `XC/RESUME` invocation. `7XF LS -O` capture is deliberately one command's output, not general stdout redirection. These are design boundaries, not a claim that XC has shipped. XC return path and 7XF output boundary
- The conceptual resemblance to the earlier QLC ambition is useful for our review; the sources do not prove an uninterrupted implementation lineage. VMEX/ASLIX as a future playground or native command environment remains separate from the current CP/M commander/catalog work.

4. How it evolved

250825 / 250828
PICO9918, F18A and the search for an existing answer

Source: private PICO9918/F18A transcript. Its filename says 250825; the recovered material also contains a 250828 label and later recovery context. We retain the interval rather than inventing a single exact session.

- The immediate wish was a ready-made display with simple character RAM and **per-cell colour RAM** for a future 7xsys host. Six-pixel characters were attractive; sixteen colours could be enough for an interim solution. The host itself was not already a finished machine.
- The search moved between TMS9918-style text/graphics, PICO9918 and F18A. Forty-column text, graphics modes and eighty-column extensions offered different compromises in font size and colour-addressing granularity.
- Early assistant answers claimed no T80, fixed palettes and missing enhanced facilities. Later parts corrected several of these claims and discussed palette RAM, GPU/DMA, scanline/sprite configuration and RP2350 variants. This is a record of a changing understanding, not a reliable feature table for one firmware revision on one August date.
- T40’s 6x8/global-colour behaviour and G32’s 8x8 patterns were compared. A colour-group explanation was corrected to groups of eight patterns; earlier per-pattern sample C code was not thereby validated.
- F18A’s enhanced per-position attributes were closer to the desired model. PICO T80 and Graphics I were discussed; T40 support remained unclear at the checkpoint. An eighty-column font-width claim also changed from eight to six.
- MSX V9938/V9958 TEXT2 blink tables were not the straightforward arbitrary foreground/background-per-cell facility we were seeking. F18A is not simply an MSX2 video chip; emulator expectations based on that equivalence failed.
- TI Extended BASIC access and helper routines for enhanced modes were explored, without a tested 7xtgv backend emerging from the surviving record.

- The physical PICO9918 v1.2 board was designed for a legacy 5-V setting. We wanted a universal 3.3-V/7xbus arrangement and asked about a 3V3 pad. The archive does not establish a verified rewiring or undervolting method.
- Outcome: we had not found an exact ready-made match **within this search**. That explains continued design work; it is not a claim of worldwide novelty.

260101-labelled archive recovering the actual QL raster

Source: QL font extraction discussion. This is a reconstruction prepared on 261013, with generated headers and images in its companion archive; it is not a certified verbatim log of one January day.

- The actual **6x10-pixel cell was not clear at first**. We moved through 6x9 assumptions and interpretations of a 96x54 image before recognising a 96x60 sheet arranged as sixteen by six glyphs of 6x10 pixels.
- A crop had omitted a top row. Incorrect grid placement, row count and glyph alignment then produced plausible-looking but wrong byte arrays.
- The assistant sometimes substituted familiar 5x7 glyphs rather than faithfully extracting the supplied QL image. Packing also drifted between low-bit and high-bit conventions. These were substantive errors, not cosmetic differences.
- Centre-pixel sampling and then multiple-sample/majority approaches were attempted. The useful lesson was to inspect a rendered glyph grid before trusting a long initializer dump.
- The target material covered 96 characters, `0x20..0x7F`, at ten bytes each. Five active columns plus spacing, blank rows and descenders had to be checked against the actual image rather than inferred from the name “QL font”.
- Manual corrections and another conversation helped, but a uniquely verified final header is not identified in this private archive. Its five generated headers are attempts, not five interchangeable authoritative font sources.

- VS Code/Notepad++ rectangular or column editing is another practical memory cue from repairing the arrays.
- The surviving 260118a sketch still declares `CELL_H 9`; the 260120 discussion reports the corrected 6x10 display working. Thus the label 260101 must not be used to claim that every later experiment already used the final raster.

260117
CircuitPython, TileGrid and the three-plane idea

Source: displayio discussion and later annotations.

- We considered translating an ESP32-C6/ST7789 Arduino HelloWorld into CircuitPython. A growing tree of labels was unattractive; the aim was a coherent blitted display without Python loops painting individual pixels.
- Native `displayio.TileGrid` looked useful because arbitrary cell dimensions, including 6x10, and integer `Group` scaling fitted the idea.
- One monochrome grid did not provide arbitrary independent colours per cell. A layered construction was discussed: black base, four paper layers and sixteen ink layers, twenty-one layers in all. This was an approach under consideration, not evidence of the later native engine running in Python.
- An initial character/attribute format used `GGPPIIII`: two glyph-bank bits, two paper bits and four ink bits. Four 256-glyph banks could be represented by a 1024-glyph sheet, for example 192x320 pixels at 6x10.
- Independent paper and ink patterns then became the useful conceptual split. A 4096-glyph/sixteen-page possibility was also discussed. Sixteen-byte storage slots imply 16 KiB for 1024 glyphs or 64 KiB for 4096, separate from the tighter row-packed budgets in the original proposal.
- Shared 1-bpp source glyphs with opposite transparency interpretation avoided requiring a separately stored inverted font sheet. **Paper draws where its mask is zero; ink draws where its mask is one.**
- Consequently SPACE/all-zero is solid paper on the paper plane but empty ink on the ink plane. An all-one paper mask cuts the paper away. This inversion is essential to remembering how the later two-pattern cell arose.

- Both a four-byte variant with separate attributes and a three-byte compromise were explored. A 52x18 grid at four bytes is 3744 bytes. This early four-byte proposal must not be mistaken for the later `CHAR_A/CHAR_B/ATTR/EXTA` layout.
- Four pastel papers, remembered as WRGB, were enough to suggest selection, error, cursor and window states while keeping sixteen ink choices.
- Manual refresh, `auto_refresh=False`, roughly 15-Hz scheduling and immediate dirty-cell updates were considered. Duplicate shadow state was something to avoid unless it solved a real rendering need.
- Arduino_GFX 1.6.4 bitmap paths also entered the discussion: indexed and opaque monochrome rectangles versus slower chromakey paths. These were version- specific observations/proposals. A small RGB565 cell staging buffer was a portable way to compose the masks correctly.
- ST7789 GRAM could hold the displayed image while we updated rectangles. A full 320x170 RGB565 buffer is 108800 bytes; at 40 MHz the ideal payload alone takes about 21.8 ms. That arithmetic is not a measured frame rate.
- Controller vertical scrolling is normally a full-width operation, not an arbitrary movable GUI window or horizontal scroll. Atari XL **ANTIC**, fine scrolling and display lists supplied important comparison ideas. Portable software copying and optional backend acceleration remained separate paths.

260118a
the first visible composited Arduino experiments

Source: Arduino experiment transcript; its ZIP contains sketches and photographs, making it more than a retrospective narrative.

- Arduino_GFX and Adafruit_GFX were compared for common bitmap primitives. GFXfont background/opaque-text behaviour was one reason not to equate a library's text API with the intended cell compositor.
- The archived demo uses a black base, an inverse paper pattern and a positive ink pattern. It has `paperRAM`, `charRAM`, `attrRAM`, an RGB565 cell buffer, `CELL_W 6`, `CELL_H 9`, and zoom 1..4.

- There is also an internal mismatch: the archived font array has four banks of 128 characters while a comment speaks of four by 256. This is a concrete reason to distinguish recovered code from the surrounding proposed format.
- Initial black-screen debugging involved waiting for Serial and nested `startWrite`/`endWrite` transactions. A first visible blit at `(20,20)` and removal of outer transactions are useful bring-up milestones in the record.
- The small ST7789 panel appears with 170/172-by-320 geometry details. An NV3007 428x142 panel was another pocket-terminal possibility, not an implemented port demonstrated by this archive.
- Early zoom reduced the available grid. Later thinking separated a logical 80x24 or 96x48 surface from the visible scaled/panned area.
- Photographs show multicoloured text and pastel papers at enlarged scale. Some later hand-tuned palette/code changes were missing during recovery.
- ADM3A, VT52, VT100 and CP/M compatibility entered as useful applications of the display. The editor/native GUI idea already existed before this terminal direction became the main implementation work.

260118b
API cleanup, palettes and SPI surprises

Source: ST7789 and palette discussion.

- A shared `gfxsetup.h` was proposed to make examples use a common board setup. An upstream contribution was an option discussed, not a completed PR here.
- Panel offsets and GRAM dimensions were a source of configuration mistakes. Values from one module cannot be treated as universal ST7789 properties.
- U8g2 fonts, MUI and small monochrome controllers were considered as nearby tools. An assistant claim about two-bit antialiasing is not established by the archive; Arduino_GFX remained the colour-demo backend in this thread.

- The palette moved towards paired shades in eight colour families for sixteen inks, with grey/silver giving extra neutral choices and WRGB pastel papers. Darkening rather than thickening a glyph remained relevant to “bold”.
- `Print`/`snprintf` alignment and a `pixel_margin` experiment for stylised scaling were practical details, not the final native text API.
- ESP32 SPI tests did not support the assumption that DMA must be faster. A blank DMA result was followed by a working explicit `spi_num 2` setup; a small counter case was about three times slower, while another larger comparison was 1212 versus 1143 ms. These are individual reported trials, not a general performance ranking.
- Changing the requested SPI frequency did not always change the result as expected. The record does not establish one definitive host/driver cause. Sharing LCD/SD SPI with separate chip selects was also discussed.
- UTC/`gmtime` handling appears as a small side topic: retained here as a cue to the surrounding working session rather than a native-video requirement.

260119
a malformed font header, distinct from wrong font pixels

Source: font-header debugging transcript.

- Generated initializers were missing an outer/bank brace structure for a `font[4][128][CELL_H]`-style declaration, causing a flood of compiler errors.
- A standalone constant ten-row glyph representation helped isolate the syntax problem. Correct C array nesting still did not certify that the extracted pixels were faithful to the original QL font.
- These two failures should remain separate in memory: a header that compiles can contain a wrong font; a correct glyph can sit in a malformed initializer.

260120
corrected 6x10 text and QL-like drawing semantics

Source: working-font, palette and text discussion.

- The conversation reports 6x10 working. Paper 0..3 and ink 0..15, `at`, `cls` and printing were explored as a compact QL-like interface.
- Edge behaviour was explicit: stop at the edge rather than silently wrap, with CR/LF handled deliberately. Clearing with SPACE and a zero paper/ink attribute was part of the implementation discussion.
- Names such as GN/DGN were used to avoid confusing green with grey while tuning light/dark pairs. Syntax-coloured C was a motivating visual example.
- An inline `€` escape, entered as AltGr+E, was proposed by analogy with VT52 controls, with CP/M and FUZIX in mind. It is a historical idea, not an adopted protocol in the current terminal or a signed native command encoding.
- Random “7xtgv cheers” output avoided the first-row counter. Photographs of small IPS pixels introduced moire, subpixel and white-balance distractions; optical zoom versus macro was discussed when judging the result.

260120/260121
RP2350 portability and the cost of the toolchain

Source: RP2350 bring-up transcript. The file says 260121, while the reconstructed conversation labels activity 260120.

- The expectation was that a common graphics API would make the ESP32-C6 demo portable. `Arduino_HWSPI` constructor arguments already broke that assumption: an argument expected `SPIClass*`, not a pin number.
- Software SPI worked; generic hardware SPI produced a black screen; an official Waveshare path produced white. These were different observations, not one proven hardware fault.

- Speculative pin-mux/core-rewiring detours and unnecessary `extern` advice complicated the investigation. A malformed multi-line `DEV_DEVICE_INIT` macro with preprocessor conditionals caused another avoidable failure.
- The recorded working path used `Arduino_RPiPicoSPI`, avoided redundant explicit SPI setup, and made initialization an ordinary function. Both the C6 and RP2350 were then reported working. A universal root cause for every failed generic-HWSPI trial was not demonstrated.
- A graphics adapter and runtime descriptors remained attractive for switching between Adafruit/Arduino backends without rewriting the native model.
- Long builds and upload cycles fed the VMEX/ASLIX discussion: a portable, more directly programmable environment could shorten that loop. The RV32EM-like mention is a direction cue, not a recovered VM implementation.
- Photos establish that code/text/cheers displays existed. They do not restore every later source edit or hand-tuned palette value.

260725
nukedc recovery and rebuilding our external memory

Source: full private recovery discussion.

- The task was to recover genuinely lost local work, not merely reorganise an intact project. Chat timestamps, generated attachments, snippets and photographs became evidence for reconstructing what had existed.
- The narrative describes a backup discontinuity during 2026, early work without this Git/Codex workflow, and generated chat files surviving as an accidental archive. Recovered-as-found files and later reconstructions need separate identities.
- The date **250515** appears for nukedc and has now been explicitly repeated in our 261013 recollection. Because the same recovery story refers to lost January 2026 work and a 2026 backup gap, the chronology is inconsistent as presently documented. We preserve the supplied date rather than silently “correcting” it to 260515 or inventing a second incident.

- Later references to February demos are another gap: this source set has no dedicated February transcript establishing an exact day or final source.
- The surrounding searches mention 7xmem connectors, MiSTer XSDS, Lua/PicoCalc blitting, ADM3A/VT52/ANSI and night palettes. Some appear only as search cues; the complete underlying discussions have not been recovered here.
- Procurement/Mouser history was suggested as a way to date hardware work. No orders have been inspected for this reconstruction, so there is no purchase-derived chronology to claim.
- The recovery discussion itself demonstrates why we now want a detailed HG: names, intermediate failures and small visual details can reconnect a memory that a polished feature list would omit. That is the purpose of keeping these apparently minor branches in this draft.

260727
VGA text, display controllers, and the unexpected CPU bridge branch

Source: VGA, P6 bridges and RAD750 discussion. These are historical research claims and questions; this HG is not a fresh datasheet audit.

- VGA text mode offered a recognisable model: character plus attribute memory, programmable fonts/palettes, cursor, start address and panning. The discussion revisited attribute bit 3 as intensity/font selection and bit 7 as blink or background intensity, with the restrictions that accompany those choices.
- VGA's familiar eight/nine-pixel character widths did not directly provide the desired six-pixel cell. Reusing the broad model did not mean inheriting all its layout constraints.
- For 480x320, 6x10 gives 80x32 cells. An 80x30 area leaves twenty vertical pixels for status; 80x24 occupies 480x240 and is a separate portable application choice. Those dimensions should not be conflated into one historical mode.
- At 80x30, two bytes per cell require 4800 bytes, three require 7200 and four require 9600. The memory cost changed with the format; the low-memory appeal persisted through all three stages.

- The search includes **S1D13xxx**, **SED1330/1335/1336**, **RA8875/76/77**, **Bridgetek EVE**, and small CPLD/FPGA text engines. Various character, graphics, acceleration and controller features were close, without an exact match identified in the reviewed discussion.
- RA8875's discussed native 8x16 text was a mismatch for this 6x10 goal; that does not imply every TFT controller or every software renderer was unusable. The question was how much of our compact programming model it supplied.
- **M69030** was remembered as an integrated-memory option, with 4 MB mentioned, but its PCI host interface brought a different scale of complexity.
- The conversation then widened into a possible small P6/Tualatin system: AGTL/GTL signalling, northbridge/GMCH versus southbridge/ICH responsibilities, i815, 440BX/PIIX4E, VIA Pro133T/266T, SiS630T/635T and ALi Aladdin Pro5T / CyberAladdin-T. These names are retained as research breadcrumbs.
- The desired shortcut was a minimal boot ROM, SRAM and 7xbus path rather than a full conventional PC chipset. A southbridge alone did not solve the CPU memory-bus problem in that reasoning: the remembered "northbridge is cruel" frustration belongs to this branch.
- A proposed FPGA P6 bus agent would buffer a 32-byte cache line and bridge to 16/32-bit asynchronous SRAM, with an initial UART "P" as a tiny bring-up goal. No built bridge or working P6 board is established here. Lowering the clock would not remove the bus-width/pin-count problem.
- Embedded P55C/MMX and Tualatin options were compared with MediaGX/Geode, Elan and STPC-style SoCs. RAD750/PPC750 and Mars formed another curiosity branch, not an adopted processor requirement for 7xtgv.
- In particular, **PPC750CXe L2 as SRAM/scratchpad** received contradictory assistant answers and a retraction. True mapped SRAM, locked cache and diagnostic access need distinct manual evidence; the archive does not establish the desired feature. The RAD750 tangent does not prove it either.

- Why retain this here? It records the larger desire for host independence and manageable buses. It helps explain the appeal of a small detachable video engine without turning every CPU mentioned into a promised 7xsys board.

260906 onward
recovery becomes sustained implementation

- September restarted from recovered material with Codex and a repository-based workflow. The initial RunCPM baseline was the recovered January 2026 code, not a claim of parity with the latest upstream RunCPM.
- The old renderer could immediately support meaningful terminal work. CP/M applications exposed real input, scroll, colour, storage and portability problems; fixing those produced the current terminal/test environment.
- In parallel, the native design became more explicit. The existence of terminal progress must not hide that the native renderer remains a separate unimplemented objective.
- The following daily record condenses dated checkpoints. For individual source links, exact commands, targets and acceptance limits, use the Features Timeline and Design evolution. A day's entry is not a claim that every feature was accepted on every board that day.

260906 — recovered baseline

- Recovery preserved the sketch, headers and provenance/manifest evidence. RP2_2 compilation used the recovered RunCPM baseline with Arduino-Pico 5.4.4 and SdFat 2.3. The important step was a reproducible starting point after the interrupted earlier work.

260907 — RunCPM and a buffered terminal meet

- The integrated path combined RunCPM, a buffered VT100-style terminal, a TFT viewport and PC mirroring. Guest tests and the working CP/M drive supplied a concrete development loop; commit `900ee6b` is an early checkpoint.

260908 — task and core separation

- ESP32-S3 work progressed from a headless test to integrated FreeRTOS tasks; RP2350 used its dual-core arrangement. CPU/terminal/storage ownership began to matter as much as drawing the characters.

260909 — measured memory placement

- W535/ESP32-S3 IRAM and interpreter measurements tested where hot code and state should live. The explicit VMEX memory lessons were recorded rather than assuming that moving code or using PSRAM necessarily improves speed. Interpreter-porting memory

260910 — profiles, benchmarks and the native four-byte brief

- Application/history banks and INI policy developed alongside VTCPU, VTDRV, VTTERM and RAW-path benchmarking. These separated processor, storage and terminal costs instead of treating one elapsed time as “CPU speed”.
- The native v0.1 and SEMI briefs specified the direction described below: two character/mask bytes, ATTR and EXTA. They were design work, not a completed native engine hidden inside the terminal.

260911 — local composition and future CLI clients

- Local input composition, profile state, persistent BUF data and deferred REBOOT handling advanced the terminal workflow.
- The DDT/Termite/Tiny BASIC plan also discussed input origins, filtered recall, future VMEX/ASLIX CLIs, favourites and scripts. It deliberately reached beyond the implemented subset. History and CLI plan

260912 — retained graphics and an overview proposal

- GCLS/xterm256 work introduced retained graphics concerns; W535 rotation, timing and start/stop helpers received attention.

- The bird’s-eye proposal considered 6x10 normal cells and 3x5, 2x3, 1x2 and 1x1 overview representations. The smallest are occupancy summaries, not legible miniature fonts. The scope was one active terminal, with future multiwindow/VMEX ideas explicitly outside that implementation. Viewport and bird’s-eye proposal

260913 — practical guest editing and transfer details

- Blink timing around 250 ms was tested on S3. TE’s eighty-column source constraints, CRLF plus Ctrl-Z guest text, SUBMIT `$1` and VTPAD work exposed the small conventions required for a usable CP/M development loop.

260914 — persistent display control and another board

- Backlight persistence, the W550 addition and REFLASH/UNSUB/SYNC workflows broadened the usable system. Device state and safe deferred persistence were becoming explicit parts of the architecture.

260915 — zoom, history and completion questions

- Zoom 1..4 and logical geometry were initially coupled to persistent settings. Retained history grew to a 255-entry configuration with filtered recall.
- BBC keyword/TOK export and selectable completion sources were discussed. Exported token data did not by itself implement automatic keyword loading.

260916 — separate compatibility axes

- The legacy compatibility design distinguished terminal output interpretation from keyboard encoding and local editing. Turbo Pascal/TINST combinations illustrated why ANSI output and VT52-style input could be useful together. Mentions of ADM3A were not proof of an implemented ADM3A backend.

260917 — ANSI/VT52 and guest-program checks

- ANSI/VT52 handling, Console 7/8-bit C1 choices, mode helpers and presets gained concrete tests, including BBCLOWER smoke evidence. Compatibility became a set of explicit choices instead of an application name standing for everything.

260918 — native desktop ideas remain separate

- Turbo Pascal/BBC profiles and DEC graphics were refined. DEC line drawing belongs to the compatibility renderer; it does not implement native WIND.
- The native discussion considered a 64-KiB, 204x80 cell scene, zoom/panning, bird’s-eye navigation and software-owned windows. DEC double-size text and native display zoom were also kept distinct. Profiles and native design notes

260919 — source boundaries and a comparator

- The Linux POSIX RunCPM comparator supported guest-program checks, while source-boundary work clarified CPU, console, terminal and rendering flow. That comparator did not exercise the actual TFT/SDL terminal renderer.

260920 — physical keys and layout reality

- Keyboard work moved towards physical key identities and INI overrides, including Czech layouts and keypad distinctions. The host terminal’s report became part of the evidence, rather than assuming that every missing chord was a firmware parser bug.

260921 — bounded configuration diagnostics

- INI errors were made inspectable through `7XTGVTRM.ERR` and bounded reporting. Invalid configuration needed useful diagnostics without overwhelming startup or silently changing unrelated settings.

260922 — key tokens and feature tracking

- Private eight-bit key tokens, Turbo Pascal proof and key-first configuration names developed. The feature registry began separating ideas, implementation and acceptance, a distinction especially important for native plans.

260923 — dependable transfers and clearer manuals

- VTPUT/VTGET replaced the unsuccessful XMODEM route; UNZIP 1.57 and guest-text conventions supported the card workflow. Transfer cleanup restored the original profile and RAW state, with side-effect-free profile reading.
- Palette/index interpretation and the `irgbscmy` naming direction were recorded. AsciiDoc/manual work made the growing behaviour easier to inspect.

260924 — board identity, palette tests and live settings

- Board-specific backlight curves, startup ramping, target identification and W535 minimum brightness received attention. BASIC ASCII fixtures, ink labels and DEC’s 32-glyph charts made visual checks reproducible.
- A live-geometry overwrite motivated fetching the device INI before editing it. A stale host copy was not a safe substitute for current device state.

260925 — RP2350 porting and persistence ownership

- RP2_2 defaults were corrected and RP2_147 was added, with per-board backlight handling and two-axis viewport work.
- INI saving used an in-memory snapshot and sequential operations instead of interleaving access through a shared SdFat cache; guest edits invalidate the relevant cached state. This was an ownership issue, not just a faster write.

260926 — logical scenes and 7XF

- Temporary zoom became separate from persisted logical geometry. VTINFO and VTZOOM made that distinction observable; keyboard zoom no longer needed to rewrite the INI.

- The catalog/LS/LL/CD direction developed under the 7XF name. The XC return proposal and limited listing-output capture kept future commander ambitions separate from a generic redirection or resident-windowing claim. XC and output boundary

260927 — catalog checks and guide roles

- Catalog formatting and paths were verified. User, Reference and Technical Reference guides acquired clearer roles as the system outgrew one manual.

260928 — emergency escape and core boundaries

- Ctrl+Alt+C emergency COM abort reached the CPU loop and also cancelled SUBMIT work. GCLS retained-image behaviour was corrected.
- Source directories were reorganised and a RunCPM core-delta audit began, separating necessary terminal hooks from unrelated inherited/core changes.

260929 — an explicit gap in this daily record

- No separate dated checkpoint for this day was identified in the reviewed material. It falls within the surrounding audit/development interval; this is a documentation gap, not a claim that no work happened.

260930 — recovered core cleanup

- Board dispatch and explicit configuration flags were clarified during the recovered-core audit. Future upstream/CP/M 3 investigation remained a plan; successful CP/M 2.2 operation did not establish upstream parity.

261001 — complete board HALs and W535 storage

- Persistence/runtime ownership and plain CCP versus 7XF responsibilities were tightened. The preferred HAL organisation became one complete, readable board header, with searchable optional XTGV hooks; fragmented headers and code-body macros were rejected during that review.

- RP2_147, RP2_2 and W535 build/boot evidence was recorded separately. W535 SPI trials encountered an SPI2/QSPI conflict, then used SPI3 and SdFat after a slow Arduino SD path. Clock trials progressed through 12, 20 and 40 MHz.
- One quiet 40-MHz VTDRV case verified about 7.95 KiB/s write and 19.72 KiB/s read. These are that setup’s results. SPI became the default; the remaining older SDMMC route was removed later. VOL/INFO reporting also received work.

261002 — W550 and P4 display/storage bring-up

- W550’s SdFat 40-MHz path gained acceptance. P4 DSI/SD bring-up needed the missing LDO4 power enable; a software build alone could not reveal that electrical dependency.
- Os/O2/O3 trials, buffered P4 SPI and configurable clocks were measured. CARDMARK read tests were not treated as write qualification; DSI underrun clock changes had their own acceptance boundary.
- Backlight minimum/PWM behaviour, OFF controls and startup blanking were practical display-quality work alongside the throughput investigation.

261003 — diagnostics, memory budgets and startup

- CARDMARK’s CSD/CMD6 query coverage broadened. A boot T-state count was corrected from 14517012 to 13015509, retaining the importance of measurement definitions rather than just publishing a larger number.
- Branding/build documentation and boot fixes accompanied an RP history configuration reduction from 255 to 100 entries, freeing about 193.75 KiB.

261004 — C6 and predictable tooling

- C6’s single-core FreeRTOS arrangement and shared-SPI locks received initial checks. Optimisation policy was made consistent across six targets.
- Quiet target-aware rebuild/reflash launchers, mirror ownership, target checks and explicit setup improved the everyday development loop. Rebuilding and uploading remained separate actions.

261005 — host setup and the real desktop direction

- Arch/Debian/Fedora setup scripts were prepared; that was not complete fresh-install acceptance on all three systems.
- Linux desktop work moved towards running the actual Arduino_GFX sketch and host filesystem adapter, rather than making a separate look-alike terminal.

261006 — six real HALs on Linux SDL

- The six board configurations ran through the C++/SDL desktop path, with host SD/File services, PTY mirroring and output-only SDL windows. EXIT/REBOOT and launcher lifecycle became part of the emulated-device behaviour.
- Optimised versus unoptimised builds produced a large speed difference; backlight presentation and GCLS ghost cleanup also improved. Desktop tests exercised shared code but could not certify physical SPI or panel timings.

261007 — lifecycle, canonical card and QL channels

- Focus/stacking/lifecycle behaviour was tried and revised; visual acceptance remained narrower than code-level tests. VTESTS ran real MBASIC/scroll cases.
- The canonical `cpmcard/` policy expanded from a selected seed to the complete working tree except private directories. QL channel research explicitly returned attention to the future native interface. QL channel research

261008 — working-card organisation and licensing boundaries

- Guest tools/languages/data were organised into A0/A1, B1 MBASIC, B2 BBC BASIC, reserved B3 COMAL, B4 Turbo Pascal, B5 FAND, B6 dBASE, and C1/C2/C3 sources, graphics and benchmarks. LS/LL sorting and display-case choices avoided renaming the underlying files merely for presentation.

- Original terminal code took an MIT licensing direction with third-party components kept distinct. This did not silently decide the licence for a future native specification/HDL or for VMEX/ASLIX.

261009 — application slots and exact history limits

- Catalog browsing, exit/clear aliases and filtered-history behaviour were checked. Retained records became eighty characters including the marker, hence seventy-nine text characters; active input capacity is a separate limit.
- Profiles became INI-owned movable slots with neutral defaults and an explicit CCP-bank assignment, rather than slot numbers implying application behaviour.

261010 — Windows desktop and measured storage overhead

- Six guide filenames/roles were established. Windows/MSYS2 work added the MinGW/Mintty/loopback path and Unicode-path handling.
- A coarse Windows sleep interval was replaced by a waitable-timer approach; one measured boot changed from about 1.23 to 0.31 s. Dirty-frame drawing and local-card versus shared-card costs were also investigated.
- Native operation probes located expensive repeated opens. Desktop storage gained a bounded cache of at most eight backing descriptors while preserving fresh logical streams and close-time flushes. This was a host optimisation, not an unmeasured change to physical SdFat behaviour.

261011 — real Windows keys and exact recall

- The user reported fixed SDL scale and Alt+Q working across six Windows variants. History insertion preserved spaces exactly, including filtered recalls, instead of manufacturing a helpful-looking separator.
- A focused Mintty helper supplied physical Czech top-row/keypad identities. MSYS-to-Win32 environment propagation lost the setting despite correct Python configuration; explicit per-window handoff fixed the reported case.

- `XTGV_HACK_TOPROW` policy, keypad limitations and layout acceptance remained explicit. A WezTerm trial exposed unacceptable colours and was later removed; future board profiles and WebAssembly were also discussed.

261012 — browser runtime and publication

- WezTerm was removed. A fresh Windows setup/rebuild/launch was reported for one unspecified target; Wine also supplied a bounded check of an actual Windows executable. Neither meant universal host/platform qualification.
- WebAssembly work progressed from a native bridge to standalone browser input, canvas display, frozen PNG/text and card ZIP handling. The six board descriptors could be selected while retaining card state.
- Worker isolation, Pages configuration permissions and publication were addressed; successful publication was reported. Guides gained a browser-first entrance. MacBook Fn/modifier observations remained partial: Alt+Esc worked in the reported case while Alt+Space did not.

261013 — readable guides and this memory reconstruction

- Guides adopted collective wording and consistent Alt labels. Shared keycaps grew slightly and were repositioned; JavaScript marked Ctrl, Alt, AltGr, Shift and Fn for lighter backgrounds. Pale-red tables, blue heading strips, black heading text and padded column legends support quick visual scanning.
- Hardware-page links were reviewed; the historical W535 software identifier was deliberately left as-is after the mismatch was acknowledged.
- HG became this detailed raw reconstruction, with source context, ecosystem context, raster uncertainty, daily checkpoints and native-SPEC review prompts. This entry records documentation work, not native-engine implementation.

5. The native format what changed, and what is still only designed

Sources: 260910 native v0.1 specification/implementation brief, 260910 SEMI and image-demo brief, and 260918 native scene notes.

Three different “small cell” stages

Stage	Stored information	Meaning and evidence
250529 draft	Character plus attribute, two bytes	Banked glyphs; three-bit paper and ink; programmable 256-colour ambition.
January demos	Paper character, ink character, attribute, three bytes	Black base, inverse paper and positive ink; GGPPIIII-style attribute. Archived January code uses 6x9; later legacy source uses 6x10.
260910 native proposal	CHAR_A, CHAR_B, ATTR, EXTA, four bytes	Mode-dependent two-mask composition, colours and brightness, or SEMI packing. Detailed design exists; the full native engine remains unimplemented.
Current compatibility terminal	paperRAM, charRAM, attrRAM, flags arrays	Four arrays do not establish native EXTA. Flags carry terminal properties such as underline, reverse, graphics and blink.

- At 80x24 the proposed native cell RAM is 7680 bytes, before glyphs, palettes, desktop storage or software window backing buffers.
- The old two-/three-byte budgets, January’s alternative four-byte attributes, and the final proposed EXTA layout should remain named separately. Simply remembering “we already had four bytes” would conceal a real design change.

TEXT, WIND and INST composition

- `ATTR = MM PP IIII`: two mode bits, two paper bits, four ink bits. Modes are `00 TEXT`, `01 WIND`, `10 INST`, `11 SEMI`, with mode-appropriate resources and palettes.
- In normal composition, A=1 selects ink; A=0/B=0 selects paper; A=0/B=1 reveals the underlying desktop. Thus text over a desktop uses a text A mask and an all-one B mask; a transparent blank uses zero A and all-one B.
- `EXTA` supplies four-bit paper and ink levels: zero black/off through fifteen nominal. **Brightness is not alpha.** A perceptual lookup is possible; a linear multiplier is not mandatory, and this does not dim the independent high- resolution desktop image automatically.
- Six glyph bits occupy row bits 7..2, with low bits reserved. This supersedes the original suggestion to pack brightness into those unused row bits.
- TEXT proposes a 256-code namespace: ASCII at 20..7E, visible/debug control representations at 00..1F, UI arrows at 80..9F, maths at A0..BF, blocks at C0..DF, editor symbols at E0..EF and reserved F0..FF. Extended assignments remain provisional; no national-character standard is silently implied.
- WIND's `TTFFSSSS` organisation gives four themes, four families and sixteen variants/states. Single, double, rounded and bevelled themes; frame NESW topology, separators/tabs, scroll/sliders and decorations were considered. Procedural geometry should join without seams at the chosen raster.
- INST's `CCCC VVVV` gives sixteen instrument classes by sixteen states: lamps, bars, knobs/meters, seven-segment-style indicators and related compact controls. Procedural generation is possible; a full ROM need not be assumed.

SEMI
a different use of the same four bytes

- A 6x10 cell becomes a 2x5 array of **3x2-pixel semipixels**. An 80x24 cell scene therefore carries a 160x120 SEMI image.

- `ATTR = 11CCAABB`: two upper mode bits, two high palette-index bits, and two high mask bits for each of A and B. CHAR_A and CHAR_B hold the corresponding eight low mask bits. EXTA supplies the low eight palette-index bits.
- A 1024-entry table with four RGB565 colours per entry takes 8192 bytes. Together with an 80x24 four-byte scene this is 15872 bytes, excluding other engine resources. The old global-256-LUT ambition and this local-palette proposal are related colour goals, not identical formats.
- Mask numbering is row-major: bits 0/1 at the top through 8/9 at the bottom.
- In `SEMI_OVERLAY`, AB=00 reveals desktop, 01 gives B, 10 gives A and 11 gives

  A. Here B is a **positive** mask, unlike the inverse paper mask in normal text composition. Losing that distinction would change the image.
- In `SEMI_INDEXED4`, AB=00..11 selects one of four colours. Overlay versus indexed interpretation is a global/surface choice in the proposal, not a fifth per-cell byte or an unallocated extra attribute bit.
- SAFE14 first resizes/quantises to fourteen global colours, then selects up to four per cell. There are C(14,4)=1001 four-colour combinations, fitting the 1024-entry palette table. Exact reproduction applies only where a cell has at most four colours after quantisation; the name is not a guarantee for arbitrary source pixels.
- An ADAPTIVE1024 route would cluster local palettes; optional dithering and quality review were later choices. The proposed parrot/image demonstration was to compare reference, encoding and decoded preview, with versioned output carrying dimensions, mode, four planes and palettes. The brief is not proof that this converter and native on-device demonstration were delivered.

Desktop, windows and the implementation sequence

- The high-resolution desktop lies outside cell RAM and may be generated, externally stored or supplied by a framebuffer. This is a resource decision still needed for each backend, not “free pixels” provided by four bytes/cell.

- Software owns window buffers, position, clipping, z order, focus and titles, then flattens the visible scene. Hardware need not contain a window manager.
- An effective window level can be combined with the cell level using a rounded `cell_level * window_level / 15` rule. Optional future alpha/fading is separate from this brightness operation.
- The FPGA/CPLD direction favours counters, RAM reads, mask selection and palette lookup. SEMI need not consume a 1024-glyph bitmap ROM. External SRAM versus internal FPGA RAM is part of the resource tradeoff.
- The 260918 candidate of 204x80 cells consumes 65280 bytes, leaving 256 bytes in 64 KiB. At 6x10 it covers 1224x800, leaving a 56-pixel horizontal remainder on a 1280-wide panel. This is a candidate scene, not a normative mode.
- That note considers 204x80 at 1x, 102x40 at 2x, and explicitly 50x20 at 4x covering 1200x800. The last is not an exact quarter of 204 columns; clipping, margins and partial-cell policy need a decision rather than rounded claims.
- Bird’s-eye navigation, focused zoom, a portable 80x24 application view and one-pixel smooth presentation scrolling are ideas outside the basic cell bytes. A Hyprland-like windowing analogy did not establish a running WM.
- The planned sequence was a clear native specification, a reference renderer, glyph/resource generation, image conversion and then on-device proof, while keeping the useful compatibility terminal. The surviving design/task briefs do not substitute for completing those steps.

6. Review prompts for the final SPEC

These are questions extracted from the history, including newly identified cross-checks. They are not additional promises or claims of implementation.

- **Purpose:** does the final document still lead with editors, windows and small hosts, while treating the terminal as one client? Is the 7xsys/7xbus and future VMEX/ASLIX boundary intelligible without requiring those systems to exist?
- **Raster:** explicitly fix 6x10, active columns, spacing, bit order, baseline, descenders and the authoritative font source. Audit all inherited 6x8/6x9 memory calculations and identify what is reconstructed versus verified.
- **Cell contract:** specify byte order, mode transition behaviour, palette selection, glyph-bank ownership and reserved bits. Keep terminal `flags` separate from EXTA even where both implementations use four arrays.
- **Compositor:** retain truth tables for normal inverse-paper and SEMI positive-B paths, including blank/transparent cells, overlapping masks and desktop exposure. Define black/off brightness precisely; do not imply alpha.
- **Colours:** reconcile the original programmable 256-colour goal, January's four papers/sixteen inks, native mode palettes, RGB565 output and SEMI's 1024 local palettes. Preserve positive/night palettes and colour-based bold as requirements to consider rather than accidentally discarding them.
- **Resources:** count cell, font, instrument/theme, palette, desktop, staging and software-window memory separately. Decide ROM/RAM mutability and what happens when shared glyphs are redefined while inactive windows still reference them.
- **Host interface:** the reviewed history does not finish the bus/register map, reset defaults, discovery/versioning, update ownership or atomicity/tearing policy. These are inferred specification gaps, not recovered lost decisions.
- **Windows and navigation:** separate logical scene, physical viewport, scale, smooth pixel scroll and bird's-eye summaries. Define partial-cell edges and who redraws after focus/z-order changes. Do not accidentally require the display hardware to schedule VM tasks or own application windows.

- **Input/API:** distinguish a native drawing/channel API from ANSI/VT52 parsing, physical keyboard transport, local composition and app key bindings. Decide which QL-like `at`/`paper`/`ink`/`cls` semantics remain useful. The historical euro escape is only a candidate unless explicitly adopted.
- **SEMI tooling:** specify palette construction, colour-error policy, deterministic encoding, overlay/indexed mode ownership, image headers and reference decode tests. SAFE14's combinatorial fit alone does not establish visual quality.
- **Backend proof:** separate portable reference output, SDL/browser demonstrations, MCU performance, electrical board bring-up and eventual HDL timing/resource proof. DMA, PSRAM or a larger compiler optimisation level should earn their place through relevant measurements.
- **Unfinished surrounding software:** retain pointers to LSB/QLC, VMEX/ASLIX, XC, keyword/favourite input sources and possible scripting, while avoiding an accidental requirement to implement all of them inside the video engine.
- **Missing context to recover:** 7xmod module details, final 7xmem connectors, MiSTer XSDS relationship, any surviving February demo, the final manually corrected font/palette, Lua/PicoCalc discussion and a resolved nukedc timeline. The available corpus names these; it does not supply all their answers.

7. Related technical history

Source map

- 250529 original concept, preserved as 260906-21: starting motivation and early cell/hardware sketches. The corrected filename says 250529; older links that said 250524 should not create a second origin event.
- 250529 private origin copy: surrounding ecosystem context and annotations.
- 250825-labelled PICO9918/F18A research: evolving claims, corrections, host-voltage and per-cell-colour questions.
- 260101-labelled QL font extraction: raster uncertainty, failed extraction and manual correction; retrospective reconstruction.
- 260117 displayio: layering, masks, palettes and scrolling.
- 260118a Arduino demo: actual archived sketches and photos.
- 260118b ST7789: API, palette and SPI/DMA experiments.
- 260119 font header: malformed initializer versus glyph content.
- 260120 demo: reported corrected 6x10, QL-like text and colours.
- 260121-labelled RP2350: portable backend and toolchain problems.
- 260725 recovery: lost files, surrounding 7xsys/7xmem work, date ambiguity and missing conversations.
- 260727 VGA/P6/RAD750: controller comparisons and broader host research, including corrected or unresolved assistant claims.
- 260910 native brief and SEMI/image brief: detailed intended four-byte engine, still distinct from delivered terminal code.
- 260918 native scene notes: 64-KiB scene, viewport and WM ideas.
- 261007 QL channel research: software-channel/native-engine boundary.
- 261013 imported HG planning note: retrospective structure and prompts. Its internal 2026-07-27 revision date does not prove implementation on the later filename date.
- Design evolution and key milestones: architectural decisions.

- Features Timeline: dated implementation evidence and links for the compact daily record above.
- Technical Reference Guide: current implementation and developer boundaries, rather than historical possibilities.

What the companion archives actually preserve

- The font ZIP contains five attempted headers and seven images. Those artifacts help reproduce the extraction failures; none should be promoted to an authoritative final font solely because it has a `.h` extension.
- The 260118a ZIP contains eleven entries, including `code/V4_mixworx_7xtgv_demo.ino`, `code/mixworx-7xtgv-demo.ino` and photographs. Read-only inspection found the 6x9, three-array compositor described above.
- The later legacy demo uses 6x10 and defaults to 2x zoom, retaining paper/character/attribute arrays, palette storage and the graphics-compatibility layer. That difference is useful evidence, not permission to overwrite the older archive with it.
- Several other ZIPs mainly repeat their transcript; the RP2350 archive also carries photographs, and the VGA/P6 archive includes a block diagram. An archive attachment is evidence only for what it actually contains.
- The current terminal state header provides a useful reality check: its underline/reverse/graphics/ blink flags are compatibility state, not the native EXTA brightness/mode implementation described in the briefs.